

SYSTEM ARCHITECTURE & CONTEXT BRIEF Project: Distributed AI, IoT, and Spatial-Commerce Ecosystem for Precision Agriculture (Working Title: NumiTerra / formerly Filahaty) **Author:** Mustapha Rabah and Imad Eddine **Role:** Lead Architect / Full-Stack Engineer / Master's Student in Computer Science (University of Łódź, Poland). **Target Audience for this document:** AI Engineering Assistant.

INSTRUCTIONS FOR AI: Act as a Senior Software Engineering Manager and Lead Architect. Read this entire context document to understand the scope, technical stack, and business logic of my project. Acknowledge receipt of this context, and then prepare to assist me in generating UML diagrams, Entity-Relationship (ER) models, API contracts, and implementation code.

1. Project Background & Evolution

This project is for my Master's thesis (supervised by dr hab. Liudmyla Koliechkina, prof. UŁ). It is an evolution of a previous Agritech startup prototype ("Filahaty") that I built for the Algerian market. We are now upgrading it from a basic BaaS prototype (Firebase + Native Android) into an enterprise-grade, highly scalable distributed system tailored for the European (Polish) agricultural market, incorporating "Green Deal" sustainability principles.

2. Core Modules & Business Logic

The system consists of three primary domains integrated into a unified microservices or modular monolith architecture:

- **Module A: Edge AI Diagnostics & Agronomy**
 - *Function:* Farmers upload images of crops/soil. The system detects diseases or nutrient deficiencies and crop suggestions [crop suggestions is the primary focus].
 - *Technical focus:* Custom Convolutional Neural Networks (CNNs) optimized for mobile/edge inference.
- **Module B: IoT Simulation & Smart Irrigation**
 - *Function:* Real-time monitoring of soil moisture, temperature, and automated water pump triggers.
 - *Technical focus:* Digital twin simulation of sensor networks, high-throughput telemetry ingestion, event-driven architecture, and latency optimization.
- **Module C: Geo-Spatial Marketplace & Community Hub**
 - *Function:* A peer-to-peer social feed and e-commerce platform allowing farmers to rent heavy machinery, trade crops, and share localized agronomic alerts.
 - *Technical focus:* Complex spatial database indexing (bounding boxes/radius queries) to ensure users only see data within their specific geographical farming zone. Real-time WebSocket communication for chat and bidding.

3. Approved Technology Stack

- **Frontend / Mobile Clients:** Kotlin Multiplatform (KMP) sharing business logic across iOS and Android, with UI built in Compose multiplatform for both IOS and Android.
- **Web Dashboard (Admin/Cooperatives):** React.js.
- **Backend Services:** Java/Kotlin utilizing the Spring Boot framework.

- **Database Engine:** PostgreSQL equipped with PostGIS extension for advanced spatial querying.
- **Message Broker / Event Streaming:** Apache Kafka (for IoT telemetry and async marketplace events) + WebSockets (for real-time chat).
- **Artificial Intelligence:** Python, TensorFlow (CUDA/Nvidia optimized for training), converted to TFLite/ONNX for edge execution.
- **Infrastructure:** Dockerized containers for backend and DB simulation.

4. Master's Thesis Research Objectives

1. **Architectural Design:** Evaluating the reliability of integrating a KMP client, a high-performance Spring backend, and asynchronous AI/IoT subsystems.
2. **Machine Learning:** Training custom CNNs for European crops and optimizing them for edge devices.
3. **Geo-spatial Engineering:** Benchmarking efficient PostGIS indexing algorithms to dynamically filter real-time marketplace items.
4. **IoT Latency Analysis:** Quantitative analysis of Kafka/Spring's ability to route high-throughput sensor streams.

5. Immediate Next Steps (Roadmap)

We need to design the system from the ground up using standard Software Engineering methodologies. Our roadmap for this chat session is:

1. **Requirements Engineering:** Define Actors and high-level Use Case Diagrams (UML).
2. **Data Modeling:** Design the relational database schema, including the PostGIS geographical entities, resulting in a comprehensive ER Diagram.
3. **System Architecture:** Create Component and Sequence diagrams (UML) for the most complex flows (e.g., IoT telemetry routing, Spatial Marketplace querying).
4. **API Design:** Draft RESTful and WebSocket API contracts.
5. **Implementation:** Setup the Spring Boot boilerplate, PostGIS configurations, and KMP architecture.